

Juan D'Elia N orden: 18 930/22 (Ar)

Arquitectura y Organización de Computadoras - DC - UBA

Segundo Parcial

Segundo Cuatrimestre 2024

Normas generales

- El parcial es INDIVIDUAL
- Puede disponer de la bibliografía de la materia y acceder al repositorio de código del taller de system programming, desarrollado durante la cursada
- Las resoluciones que incluyan código, pueden usar assembly o C. No es necesario que el código compile correctamente, pero debe tener un nivel de detalle adecuado para lo pedido por el ejercicio.
- Numere las hojas entregadas. Complete en la primera hoja la cantidad de hojas entregadas
- Entregue esta hoja junto al examen. La misma no se incluye en el total de hojas entregadas.

Régimen de Aprobación

- Para aprobar el examen es necesario obtener como mínimo **60 puntos**.
- Para promocionar es condición suficiente y necesaria obtener como mínimo **80 puntos** tanto en este examen como en el primer parcial

NOTA: Lea el enunciado del parcial hasta el final, antes de comenzar a resolverlo.

Contexto

Nos encargaron el desarrollo de un kernel para la nueva consola Orga Génesis que cuenta con un procesador x86. Para ahorrar trabajo vamos a tomar nuestro kernel y expandirlo para permitir la ejecución de juegos mediante cartuchos, que contendrán el código y los recursos gráficos (sprites de personajes, fondos). Se incorpora entonces un lector de cartuchos que, entre otras cosas, copiará los gráficos del cartucho a un buffer de video de 4 kB en memoria. Cada vez que el buffer esté lleno y listo para su procesamiento, el lector se lo informará al kernel mediante una interrupción externa mapeada al IRQ 40 de nuestro x86.

Primer ejercicio

Distintas tareas se ocuparán de mostrar los gráficos en pantalla y de realizar otros pre/postprocesamientos de imagen on-the-fly.

Para esto, el sistema debe soportar que las tareas puedan acceder al buffer de video a través de los siguientes mecanismos:

- DMA (Direct Memory Access) - se mapea la dirección virtual 0xBABAB000 del espacio de direcciones de la tarea directamente al buffer de video.
- Por copia - se realiza una copia del buffer en una página física específica y se mapea en la dirección virtual ~~0xBABAB000~~. Cada tarea debe tener una copia única.

El buffer de video se encuentra en la dirección física de memoria 0xF151C000 y **solo debe ser modificable por el lector de cartuchos**.

a) Programar una función `void buffer_dma(pd_entry_t* pd)` que dado el page directory de una tarea realice el mapeo del buffer en modo DMA.

b) Programar una función `void buffer_copy(pd_entry_t* pd, paddr_t phys)` que dado el page directory de una tarea realice la copia del buffer a la dirección física pasada por parámetro y realice el mapeo correspondiente. *virt*

Segundo ejercicio

Las tareas configuran en memoria una variable que sirve para comunicar al sistema de qué manera van a acceder al buffer de video.

Cada tarea guarda en la dirección virtual 0xACCE50 (mapeada como r/w para la tarea) un `uint8_t` acceso con posibles valores 0, 1 y 2. El valor 0 indica que la tarea no accederá al buffer de video, 1 que accederá mediante DMA y 2 que accederá por copia. De acceder por copia, la dirección virtual donde realizar la copia estará dada por el valor del registro ecx al momento de llamar a `opendevise`.

Cada vez que se indique como completo el buffer, se deberá mapear el mismo a las tareas que utilicen DMA y hayan solicitado acceso al buffer, o actualizar la copia del buffer de las tareas que acceden por copia y hayan solicitado acceso al buffer.

Es deseable que cada tarea que accede por copia mantenga una única copia del buffer para no ocupar la memoria innecesariamente.

Para solicitar acceso al buffer de memoria, las tareas deberán informar que desean acceder a él mediante una syscall `opendevic`. El sistema no debe retomar la ejecución de estas tareas hasta que se detecte que el buffer está listo y se haya realizado el mapeo DMA o la copia correspondiente. Una vez que la tarea termine de utilizar el buffer, deberá indicarlo mediante la syscall `closedevic`. Mientras esté en uso el buffer, la tarea deberá poder visualizar su valor actualizado, ya sea por DMA o por copia.

Se muestra el comportamiento esperado en el siguiente esquema, donde las flechas indican que la información actualizada del buffer se disponibiliza para la tarea señalada.

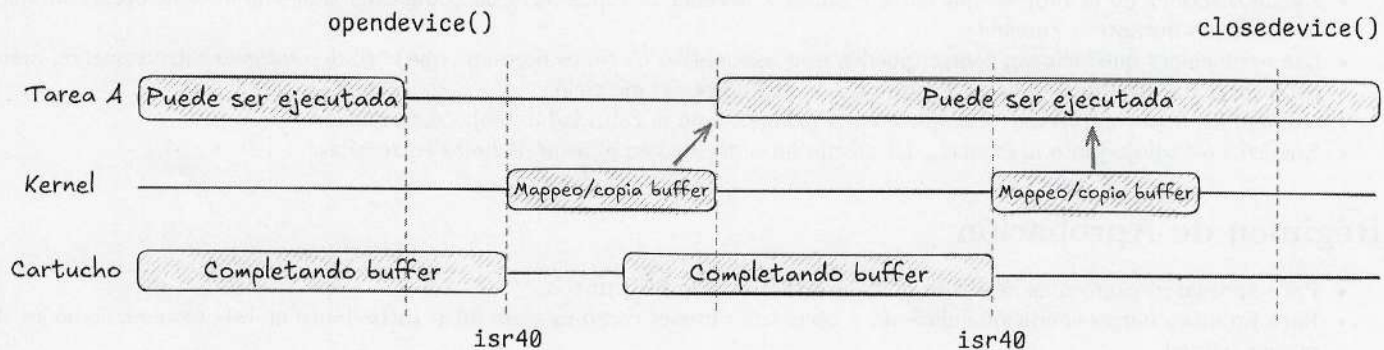


Figure 1: Diagrama schedule tarea, cartucho y kernel

Utilizando las funciones del primer ejercicio, se pide:

a) Programar la rutina que atenderá la interrupción que el lector de cartuchos generará al terminar de llenar el buffer.

Consejo: programar una función `deviceready` y llamarla desde esta rutina.

b) Programar las syscalls `opendevic` y `closedevic`.

A tener en cuenta para la entrega (para todos los ejercicios):

- Indicar **todas las estructuras de sistema** que deben ser modificadas para implementar las soluciones.
- Está permitido utilizar las funciones desarrolladas en los talleres.
- Es necesario que se incluya **una explicación con sus palabras** de la idea general de las soluciones.
- Es necesario escribir todas las asunciones que haga sobre el sistema.
- Es necesaria la entrega de código que implemente las soluciones.

Asignatura

Juan Dela

930/22

Ejercicio 1

a)

Para hacer el mapeo puedo usar la función `mmu-map-page`, que si le paso como parametro el `page-directory-entry` por su formato ~~que tiene en los 20 bits más altos la dirección de la entry~~ la macro `CH3-TO-PAGE-PAH` va a seguir funcionando. Entonces puedo usar `mmu-map-page`. (en el taller me acordé)

Voy a mapear la dirección virtual indicada por DMA a la indicada del buffer de videos. Esto debe ser únicamente modificada por el buffer así que no le doy permiso de escritura.

void

```
buffer-dma(Pd-entry-t * Pd)
```

```
mmu-map-page((uint32_t) Pd, 0x80000000, 0xF0000000, MMU_WRITE)
```

sin write!!!

Para `buffer-copy` podemos buscar la dirección física de la virt que nos pasaron por parametro y usando `copy-page` copiar lo que hay en la dirección virtual a la física.

```
void buffer-copy(Pd-entry-t * Pd, Paddr-t Phys, Paddr-t Virt)
{
    Paddr-t Src-Phys = virt-to-phys(Virt, Pd);
    copy-page(Phys, Src-Phys);
}
```

6 años

b) Parametro ~~va~~ a copiar el ~~buffer~~ del ~~buffer~~ direccion fisica indicada en

Hay que copiar el contenido del buffer a la direccion fisica indicada. Al ser ambas direcciones fisicas puedo hacer copy-page.

Por ultimo para poder usar el Page Directory y las direcciones asi que puedo usar malloc.

```
void buffer_copy(addrpd_entry_t* pd, ← direccion fisica bufferaddr Phys, vaddr virt)
{
    copy_page(Phys, 0xF151C000);
    (uint32_t)
    mmu_map_page(pd, virt, Phys, MMU_U MMU_U | MMU_P);
}
```

Si hubiera cualquier problema con respecto al pd = map_page (en mi taller funciona) se puede cambiar map_page (o hacer otra version), que tome un pd y en lugar de al principio calcularlo con CH3-to-page-dir, usar el directorio pasado por Param.

Cabe aclarar que es necesario un unico copy_page y un unico map (en ambas) xq se copian los graficos a 4 KB de memoria (una pag)

Juan D'Elia
930/22

Ejercicio 2

a) Idea: Voy a tener marcadas las tareas que hicieron ofendive, para que una vez el buffer marque listo, todas aquellas que pidieron acceso se les sea concedido y actualizado. También voy a tener que tener marcadas a las que hicieron closed para sacárselo. Al no desmarcar el pedido de acceso, ameros que haga un close, cada vez que el buffer interrumpa va a seguir actualizando a las que debe. Para esta implementación voy a hacer de cuenta (lo voy a hacer después) que tengo una lista que tiene a las tareas ~~que solicitan acceso~~ atributo a la lista, parecida a la del scheduler va a tener su selector para poder acceder a la TSS de la tarea y buscar su ^{offset} para realizar el mapeo y state que indica el tipo de acceso. Si State es 0 me tengo que asegurar de no actualizar más el buffer para esta tarea (closed marca con 0).

Además las tareas cuando hagan ofendive se van a marcar en el scheduler como "Processing" y termina el mapeo. Cada vez que ~~se~~ el buffer se complete va a marcar las Processing de nuevo como AVAILABLE en su state. Esto para evitar que se ejecuten tareas que pidieron buffer que todavía no se mapea.

También van a tener otro atributo. PhysSqr, que va a indicar en q' pag física está la copia del buffer y en virt a donde mapear.

IDENT. ENTRY ? (no)

↑ //guardo la virt y Phys. Pero que sea
 * b_tasks[i].virt = virt
 b_tasks[i].Phys = Proxima

```

  isr40:
    PUSHAD
    call pic_finish1
    call device_ready
    POPAD
    iret
  
```

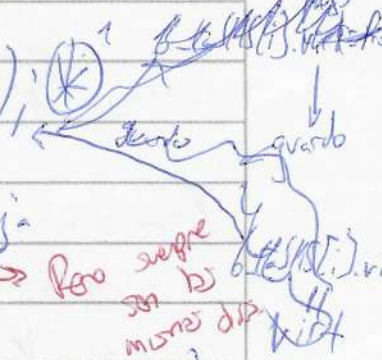
← asumo q' es de hardware ✓

//Cada vez q' termine el buffer actualizo a los tareas en mi nueva lista b_tasks

```

void device_ready():
  for(int i=0; i<MAX_TASKS; i++) {
    uint32_t cr3 = getCR3(b_tasks[i].Selector);
    if(b_tasks[i].State==0) { //caso usual copia y DMA
      Unmap(cr3, 0xBABAB000);
      Unmap(cr3, b_tasks[i].Phys); //si no está definido 0 virt
    }
    if(b_tasks[i].State==1) { //le saco la copia y mapeo DMA (?)
      buffer_dma(CR3_TO_PAGE_DIR(cr3));
      Unmap(cr3, b_tasks[i].Phys); //desmapea la virt de la copia que le
      Unmap(cr3, virt);
    }
    if(b_tasks[i].State==2) { //le saco el DMA y actualizo/mapeo copia
      //caso me hizo recién open, direc virt en ecx
      if(b_tasks[i].virt == 0) { //no tengo página virt p' actualizar
        virt = get_ecx(b_tasks[i].Selector); //está en la fila lo último q' hizo
        Unmap(cr3, 0xBABAB000); //DMA // fue open
        Proxima = next_free_user_Page();
        buffer_copy(CR3_TO_PAGE_DIR(cr3), Proxima, virt);
      } else { //sino es 0 es xq' tengo q' actualizar la copia
        Unmap(cr3, b_tasks[i].virt); //desmapeo la copia
        Proxima = next_free_user_Page();
        b_tasks[i].Phys = Proxima;
        buffer_copy(CR3_TO_PAGE_DIR(cr3), Proxima, b_tasks[i].virt);
      }
    }
  }
  for(i=0; i<MAX_TASKS; i++) { //los que estaban esperando las habilito
    if(b_tasks[i].State == Processing) {
      Sched_enable_task(i);
    }
  }
}
  
```

pero siempre se le da mismo dir virt



Sigue 2a

Juan D'Elia 930/22

Básicamente la función hace los ~~map/unmap~~ necesarios para cada tarea dependiendo de su state (sin importar hace cuanto hayan llamado a `open`).

Para esto tengo una lista con structs que (en el mismo orden que el Scheduler) tienen el tarea con su selector, state, phys y virt que guardan donde está maquina la copia. Y al final vuelve a habilitar todas las tareas que hicieron `open` (las voy a marcar como `Processing` en `open` y `close` también modificando su state).

```
uint6_t
getidx(selector) {
    uint6_t idx = selector >> 3;
    return gdt[idx].base -> cr3;
}
```

```
getecx(selector) {
    uint6_t idx = selector >> 3;
    tss_t* tss = gdt[idx].base;
    uint32_t* Pila = tss -> esp;
    uint32_t ecx = Pila[6];
    return ecx;
}
```

OK

ECX 6
EDX 5
EBX 4
ESP 3
EBP 2
ESI 1
EDI 0

Lo busco de la Pila xq al ~~open~~ momento de llamar `open` `ecx` es válido, después de eso es la interrupción de `open` ~~Además~~ ~~esto~~ ~~cada~~ ~~vez~~ ~~que~~ ~~se~~ ~~hace~~ ~~un~~ ~~cambio~~ ~~de~~ ~~contexto~~ y llamo a `Schednext`, así que los regs pueden estar mal, pero en la Pila estaban bien, así que la Pila de la `tss` vale.

Importante: `b_tss` inicializa con todas las tareas con virt y phys en 0 ~~sin~~ ~~des~~ state tmb en 0.

Siempre asumí segmentación Flat, paginación activada,
y que las tareas ya habían corrido al menos
una vez.

Juan O'Elia
930/12

2 b) open device y close device
agrega en la idt ints de nivel }
idt-intt {

IDENTITY_3(99); open ✓

IDENTITY_3(100); close ✓

...

Mantén la búsqueda en la tss en (7)
la int del buffer

En open voy a tener (si es necesario) q' actualizar
la direc virt de la página con y el state de la tarea en
b_tasks y marcarla (al open) como Procesando en el scheduler
defino en isr 99 la syscall. Además después de un
open tengo que hacer jump a una tarea q' no esté esperando al buffer.
isr 99:

Pushad

mov byte ax, [0xACE50]; busco el state (104)

Push ax

Si llega 0 va a desmontar todo

call actualizar_estado

add esp, 4

Popad

iret

esto ach no quiero volver a la tarea hasta q' termine de procesar
me voy como con el scheduler isr 32

call Sched_next_task

mov word [Sched-task-selector], ax

jmp far [Sched-task-offset]

isr 100:

Pushad

call close

Popad

iret

Salto sin importar que me
quiero ir, de última idle

void actualizar_estado(int estado){

b_tasks[current_task].state = estado; ~✓

Sched_tasks[current_task].state = Processing;

}

void close(){ b_tasks[current_task] = 0; 0 desmonta todo en isr 40

